OpenStack と AOSP プロジェクトを対象に 逆強化学習を用いたモジュール別レビュー動態予測

橋本 一輝[1,a)]　伊原 彰紀[1,b)]

概要：オープンソースソフトウェア (OSS) 開発において，将来的に開発者が数年単位で活動を継続するか否かを予測する手法が多数提案されている．従来研究の多くは，累積統計量を説明変数として Random Forest（RF）等の機械学習モデルを提案しているが，活動量の時系列的な変動を捉える手法は提案されていない．本研究は，特定のモジュールに対して特定の開発者が将来レビュー活動を行うか否かを予測（レビュー動態予測）するために，活動量の時系列的な変動に基づき学習する逆強化学習モデルを提案する．OpenStack と AOSP のコードレビューに貢献する開発者それぞれ 436 人 **TODO**: [XX 人] を対象にケーススタディを実施したところ，**TODO**: [この結論は変更なし？]RF と提案手法はほぼ同じ AUC-ROC を示したが，両モデルは異なる開発者タイプの予測に成功する補完的な関係であることを明らかにした．

1. はじめに

オープンソースソフトウェア (OSS) は，不特定多数の開発者からの貢献により支えられており，個人の知的好奇心や開発技術の学習などを動機として貢献している開発者も多い [1]．Raymond らは，OSS 開発の成功例として Linux プロジェクトを対象に調査し，不特定多数の開発者が自発的に参加する開発形態をバザール方式と名付けた [2]．この開発形態は，世界中の多様な専門知識を持つ開発者の参加を促進し，新規参入する開発者が増加する一方で，わずかな活動の後に活動を停止する開発者も多く，プロジェクト開発要員の流動性が極めて高い [3]．その結果，OSS プロジェクトの継続的な保守が困難になることもある [4]．

OSS の運用・保守は，長期間にわたりプロジェクトに貢献する一部の開発者（LTC: Long-Term Contributor）が担っていることが多く，このような開発者が活動を停止すると，実装方針，検証基準のような文書化されにくい知識の引き継ぎが困難になる．本研究で対象とするコードレビュー作業において，開発者の活動停止は，OSS の品質低下や特定モジュールを熟知する開発者の不足につながる [5][6]．したがって，将来的に発生する開発作業の依頼に対し，活動する開発者を確保できるかを見積もる手法が期待される．

[1] 和歌山大学
Wakayama University, Sakaedani 930, Wakayama-city 640–8510, Japan
[a)] s276189@wakayama-u.ac.jp
[b)] ihara@wakayama-u.ac.jp

従来研究では，後継開発者の育成や開発知識の引き継ぎに向け，プロジェクトへの貢献量（コミット・イシュー対応等）に基づき 1 年後以降や 3 年後以降の開発者の活動継続を機械学習を用いて予測する 2 クラス分類モデルが提案されている [7][8][9]．Ruangwan らは，ある特定の時点のコードレビュー依頼（以降，レビュー依頼）に，開発者が貢献するか否かを予測するモデルを構築している [10]．これらの手法は，開発者のこれまでの累積統計量を用いて将来的な活動停止の有無を予測しているが，リリースサイクルに同期した周期的な活動，集中的に取り扱うモジュールの変化，開発者の貢献停止，貢献再開など，時間的な変動を捉えられておらず，将来的に発生する特定モジュールに関する作業依頼に対し，貢献する開発者を確保できているか見積もる手法は提案していない．

本研究は，開発者への依頼と，開発者がその依頼に対して貢献したか否かを明確に観測できるレビュー活動を題材に，将来の開発者のレビュー動態（特定のモジュールに関するレビュー依頼に貢献するか否かの時間経過順の変化）を予測する逆強化学習モデルを提案する．具体的には，ある任意の時点から先の 1 年間を等期間に 4 分割し，その任意の時点で活動している開発者がそれぞれの期間で特定のモジュールのレビュー依頼に貢献するか否かを予測するために，時系列情報を用いた逆強化学習モデルを用いる．

本論文の構成は以下の通りである．2 章で関連研究を整理し，本研究の位置付けを示す．3 章で提案手法である IRL モデルの構築方法，4 章で評価方法について述べる．5 章でケーススタディを行い，6 章で結果を考察する．7 章で妥

当性への脅威を整理し，8 章で結論と今後の課題を述べる．

2. 関連研究

2.1 OSS プロジェクトに貢献する開発者の継続予測

OSS 開発者の継続性に関して多面的に調査されてきた．Iaffaldano らは一時的な活動休止の動機を分析し，私生活の出来事や興味の喪失といった個人要因と，プロジェクト貢献者間の対立やプロジェクト方針の変化といった組織要因から開発者が活動を停止することを示している [11]．Calefato らはコア開発者の活動休止の頻度と復帰する確率を算出している [12]．Vasilescu らは性別および在籍期間の多様性がチームのメンバ入れ替わり率に与える影響を分析し，新規・熟練メンバが混在する度合いが高いチームほどメンバ入れ替わり率が高まる一方で，性別の多様性は入れ替わり率に有意な影響を示さないことを報告している [13]．Lin らは生存分析により，早期参加・ファイル修正中心・コーディング中心の活動パターンを持つ開発者ほど活動の継続期間が長いことを明らかにしている [14]．これらの従来研究では，開発者が多様な理由で貢献を継続・停止している知見を明らかにしており，事前に開発者の継続性を予測する手法の確立が期待される．

開発者の継続予測モデルを提案する従来研究では，将来の単一時点以降に開発へ貢献するか否かの 2 クラス分類モデルを提案している．Zhou と Mockus は，OSS プロジェクトへ新規に参加してから 1 か月間の累積活動量に基づき，1 年後以降も継続して貢献するか否かを予測するモデルを構築した [7]．Bao らは GitHub でアカウントを持つ開発者を対象に 1 年後，2 年後，3 年後以降に同じプロジェクトに貢献しているか否かを予測するモデルを構築している [8]．Constantinou と Mens は，RubyGems/npm エコシステムで規則性 **TODO**: [規則とは何の規則？] と長期継続との関連を示した [9]．Shade らは，コードレビュー作業の依頼に，貢献する開発者を推薦する手法を提案している [10]．これら従来研究では，リポジトリへのコミットやイシューに関する貢献の累積統計量を説明変数とした機械学習モデルを構築し，数年後の特定時点以降の活動の有無を予測するモデルを構築している．しかし，活動量の時系列的な変動（増減パターン）を捉える手法ではなく，将来のレビュー動態を数か月単位で予測する手法ではない．

3. 逆強化学習を用いたモジュール別レビュー動態予測モデル

本研究は，特定のモジュールに対して特定の開発者がレビューに貢献するか否かを予測する（レビュー動態予測）ために，ある任意の時点で活動している開発者の時間経過順の行動を捉えるモデルを構築する．なお，本研究ではモジュールをディレクトリ単位で定義し，以降ではディレクトリと表記する．具体的には，逆強化学習 (IRL: Inverse Reinforcement Learning) をレビュー動態予測に適用した手法を提案し，累積統計量に基づく Random Forest (RF) と比較する．具体的なモデル構築方法は 3.3.1 項で述べる．

3.1 強化学習・逆強化学習の基礎概念

強化学習 (RL: Reinforcement Learning) は，エージェント（本研究では開発者）が環境に適応しながら将来にわたって得られる報酬の合計（累積報酬）の期待値を最大化する行動選択規則（以後，方策）を学習する枠組みであり，マルコフ決定過程に基づいて定式化される．逆強化学習 (IRL: Inverse Reinforcement Learning) は，RL を基本とする方法であるため，まず RL から説明する．

RL は状態 s，行動 a，報酬関数 $r(s,a)$，方策 $\pi(a|s)$ の 4 つの要素から構成される．状態はエージェントが観測する環境の情報，行動はエージェントが選択する操作，報酬関数は状態・行動の組に対して行動の良し悪しを数値で示す．方策は状態に対する行動の確率分布をそれぞれ表す．ソフトウェア工学分野においても強化学習を適用した研究が進められており，Liu らはバグトリアージ問題に RL を適用し，バグレポートの情報を状態，開発者の選択を行動，割り当て結果の正否を報酬，どの開発者を選ぶかの行動選択規則を方策として定義している [15]．

通常，RL は報酬関数が既知であることを前提としている．例えば，Liu らのバグトリアージでは，正しい開発者に割り当てたか否かという明確な基準を報酬関数として定義できる．しかし，本研究が対象とする開発者行動においては，レビュー依頼に対して貢献することが開発者にとって最良の選択であるかを事前に定義できない．したがって，本研究では開発者の行動パターンを正確に捉えるために，行動履歴から報酬関数を推定する IRL を用いる．IRL は，エージェントの状態と行動の時系列記録である行動軌跡 τ を観察し，その行動を説明する報酬関数を逆算する枠組みである．報酬関数 $r_\theta(s,a)$ は n 次元の特徴量ベクトル $\phi(s,a)$ と学習により推定されるパラメータ（重みベクトル） θ の重み付き和 $r_\theta(s,a) = \theta^\top \phi(s,a)$ として表現され，本研究では状態・行動の組に対する開発者の行動傾向を一つの数値で表す．なお，θ の決定方法および推定した報酬関数から方策を導く方法については 3.3.1 項で述べる．

本研究では，開発者をエージェント，開発者が観測するレビュー活動の履歴を状態，特定のディレクトリのレビュー依頼に対する承諾・拒否を行動として定義する．特徴量は開発者状態，ディレクトリ親和度，活動特徴の 3 カテゴリ 37 変数（詳細は 3.2.1 項の表 1 に示す）で構成し，IRL により過去のレビュー行動履歴からレビュアーの暗黙的な判断基準を報酬関数として推定する．

3.2 特徴量・行動軌跡・ラベルの定義

本研究では，開発者 d とディレクトリ r の組 (d,r) を予測単位とし，図 1 に示すスナップショット時刻 t_0 における

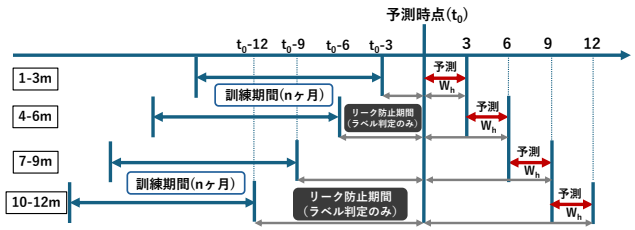


図 1: 各モデルの時間分割方法

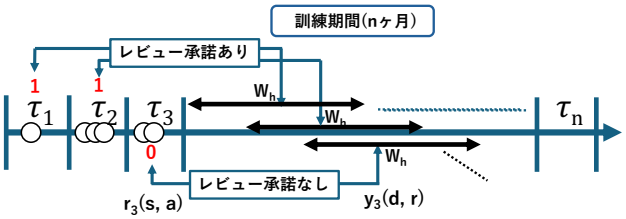


図 2: ラベル付の方法 (例：1-3m モデル)

表 1: 特徴量リスト

カテゴリ	次元数	特徴量（抜粋）
開発者状態	26	累積レビュー数・受諾率・直近性・レビュー承諾/非承諾の変動回数等
ディレクトリ親和度	6	過去レビュー数・直近性・受諾率・所有者重なり等 **TODO**: [何も変わっていない？]
活動特徴	5	行動強度・変更行数・レビュー規模・応答時間・協働の反復率

開発者の状態を起点とする．t_0 から遡る 2 年間（24 か月）のレビュー履歴を月次に集約することで，t_0 時点での開発者の状態を表現する．評価開始時点 $h \in \{0, 3, 6, 9\}$ ヶ月後を開始点とする 3 か月間を評価期間 W_h とし，W_h 内に受けたレビュー依頼に対して貢献するか否かを予測する．

3.2.1 特徴量

状態 s は，表 1 に示す通り，開発者状態（26 次元），ディレクトリ親和度（6 次元），活動特徴（5 次元）の計 37 次元から構成し，IRL と RF の両モデルで共通して使用する．なお，従来研究では評価対象のプロジェクト内の貢献のみを特徴量としているのに対し，本研究では分析対象とする他のプロジェクトへの貢献も特徴量として計測する．

3.2.2 行動軌跡

時刻 t における 1 か月間の行動軌跡を $\tau_t = (s_t, \phi_t(s_t, a_t), y_t)$ と定義する．訓練軌跡 $\mathcal{T}$ は図 2 に示すように，各月末を基準点として τ_t を時間経過順に集約することで，$\mathcal{T} = (\tau_1, \tau_2, \ldots, \tau_n)$ として訓練期間 n か月の時系列として構成する．これにより，開発者の行動変化を時間経過順に捉えることが可能となる．

3.2.3 ラベル

予測モデルの学習データのラベルとして，レビュー依頼に対して貢献するか否かのラベル $y_h(d, r)$ を図 2 および式 (1) に示す通り定義する．

$$y_h(d, r) = \begin{cases} +1 & W_h\text{内に依頼あり かつ 1 件以上貢献} \\ 0 & W_h\text{内に依頼あり かつ 貢献なし} \\ 0^* & W_h\text{内に依頼なし} \end{cases} \tag{1}$$

0^* は弱負例として重み 0.5（正例・強負例は重み 1.0）を付与する．これは PU 学習の考え方を踏襲し，依頼がない状態を強い負例として扱うと評価期間後半に貢献を再開する開発者を誤学習するため，判断できないケースを小さい重みで保持する [16]．学習に用いるラベルは月単位で，W_h 内のレビュー依頼に対する承諾の有無に基づき付与する．

3.3 モデル構築

本節では，3.2 節で定義した特徴量およびラベルを用いて構築する 2 つのモデルを説明する．3.3.1 項において，本研究で提案する時系列情報を用いて報酬関数を推定する IRL モデルについて述べる．3.3.2 項において，従来手法で頻繁に用いられる累積統計量から分類する Random Forest (RF) モデルについて述べる．

3.3.1 提案手法：IRL に基づくレビュー動態予測モデル

本節では，IRL のモデル構築手法として，報酬関数の定式化と LSTM を用いた特徴量ベクトルの計算，および開発者が貢献する確率の推定方法を述べる．

報酬関数 $r_\theta(s, a) = \theta^\top \phi(s, a)$ を決定するために，観測された行動軌跡を説明しつつ，方策のエントロピーが最大となるパラメータ θ を求める．ここで，過去のレビュー行動履歴の経験分布を $\mathcal{D}$ とすると，この問題は，過去のレビュー行動履歴から得られる報酬の期待値を最大化しつつ，方策の多様性を保つ次の最適化問題として定式化できる [17]．

$$\max_\theta \mathbb{E}_{\mathcal{T}\sim\mathcal{D}}\left[\sum_t r_\theta(s_t, a_t)\right] - \log Z(\theta) \tag{2}$$

ここで，$\mathbb{E}_{\mathcal{T}\sim\mathcal{D}}[\cdot]$ は過去のレビュー行動履歴 $\mathcal{D}$ から得られた行動軌跡 $\mathcal{T}$ に対する期待値を表す．また，$Z(\theta)$ は方策の確率分布の合計が 1 になるよう調整する正規化定数であり，$-\log Z(\theta)$ は方策のエントロピーを最大化する．エントロピーを最大化することで，未観測の行動に対しても有限の確率を割り当てることができる．

式 (2) を解くと，方策は $\pi_\theta(a|s) \propto \exp(r_\theta(s, a))$ の形，すなわち softmax 関数として表現される．softmax 関数は，貢献する確率として $[0, 1]$ で出力する．学習パラメータ θ の更新には最適化アルゴリズムである Adam[18] を用い，cross-entropy を損失関数として最小化する（学習率 $= 3 \times 10^{-4}$，エポック数 $= 150$，乱数シード $= 777$）．なお，検証データの損失が 15 エポック改善しない場合に学習を早期終了する．

3.3.2 比較手法：Random Forest を用いたレビュー貢献予測モデル

本研究では，比較対象として従来研究で使用されることの多い Random Forest (RF) を用いたモデルを構築する．RF は複数の決定木を組み合わせた学習手法であり，各決定木の予測結果の多数決により最終的な予測を行う．本研究では，3.2 節で定義した 37 次元の特徴量を用いて学習する [19]．貢献する確率は $[0, 1]$ で出力する．学習設定には，決定木の数 = 100，木の深さ（制限なし）・葉あたり最小サンプル数 = 1 等の既定値，乱数シード = 777 を用い，ハイパーパラメータ探索は行わない．クラス不均衡には `class_weight=balanced` で対処する．なお，式 (1) の弱負例重み（0.5）は IRL の損失関数にのみ適用しており，RF の学習には適用していない．

4. 評価方法

4.1 予測精度の評価方法

本研究の予測単位は開発者-ディレクトリのペア (d, r) であり，評価もこのペアを基本単位（ペア単位）とする．ただし，積極的に活動する開発者が多くのディレクトリに関わっている場合，その開発者の行動結果が評価に大きく影響するため，開発者ごとに集計した単位（開発者単位）の評価結果も示す．予測精度の評価指標として，ペア単位および開発者単位の両方に AUC-ROC を用いる．AUC-ROC は，モデルが貢献する開発者と貢献しない開発者をどれだけ正しく識別できるかを $[0, 1]$ の数値で表す指標であり，1 に近いほど予測精度が高いことを示す．

4.2 モデルが予測できるペア・開発者の分析方法

本研究では，IRL と RF のそれぞれが予測可能な開発者の特徴を明らかにするために原型分析を行う．原型分析は，データ中の極端なパターン（原型）を複数用意し，対象のデータ点がどの原型に近いかを表現する手法である [20]．

本研究では，予測実験と同様に，開発者-ディレクトリのペア (d, r) を単位とするペア単位と，開発者ごとに集計した開発者単位の 2 つの粒度で原型分析を行う．原型分析で対象とする原型数は，原型数を増やしても平均残差平方和の改善が小さくなる点に基づき選択する．なお，原型分析は予測には用いず，IRL と RF の両モデルが予測可能な開発者の特徴を明らかにする手法として位置付ける．

5. ケーススタディ

本研究では，主たる評価対象として OpenStack を用いる．OpenStack はコードレビュー管理システム Gerrit により，レビュー依頼・応答が構造化されたログとして取得でき，governance に登録されたサービスチームを単位として，リポジトリ内ディレクトリという粒度でモジュールを一貫して定義できる 32 件の主要リポジトリを客観的な基準で選定できる．また，開発者の継続予測に関する既存研究 [7][8][9] でも対象とされてきた代表的な大規模 OSS であり，先行研究との比較可能性も高い．同じく Gerrit を用いる大規模 OSS である Android Open Source Project (AOSP) を対象に追加のケーススタディを行う．AOSP は OpenStack と同様のチーム単位の選定基準が適用できないため，活動量に基づく基準でリポジトリを選定する．以下，実験設定・予測精度は両者を順に示し，原型分析以降はエコシステムごとに分けて詳細を述べる．

5.1 実験設定

対象リポジトリ（OpenStack）：評価対象は OpenStack の公式サービスリポジトリ 33 件のうち，分析対象期間にレビュー履歴が存在しなかった sunbeam を除く 32 リポジトリとする．なお，特徴量の計測においては，評価対象のリポジトリだけでなく，開発者が貢献する他のリポジトリへの活動も計測対象に含める．

データ取得と前処理：OpenStack プロジェクトではコードレビュー管理システム Gerrit を使用している．Gerrit では任意のコードレビュー依頼 1 件につき 1 つのチケットが作成され，レビュアーの指定，議論，ソースコードの承認などが管理される．各チケットにはレビュー依頼した開発者を列挙した reviewers フィールドがあり，本研究ではこのフィールドから依頼された開発者を取得する．ただし，reviewers フィールドに登録されていないが，レビューに貢献した開発者も存在するため，reviewers フィールドの開発者とレビューに貢献した開発者の和集合をレビュー依頼の対象とする．

なお，依頼されたレビュアーのうち bot アカウント（CI・自動化）は除外する．具体的には，Gerrit アカウントのユーザ名またはメールアドレスに，CI・自動化アカウントを示す部分文字列（`zuul`, `gerrit`, `-ci-`, `-test@`, `-bot@`, `@review.openstack.org` 等）が含まれるかを部分一致で判定し，該当するアカウントを除外する．この判定により 113 アカウント，レビュー依頼 75,394 行（候補行の 30.1%）を除外した．除外行数の上位アカウントを目視確認したところ，いずれもサードパーティ CI であった．**TODO**: [**AOSP は書かない？**]

また，レビュー依頼に対して開発者が貢献した履歴は，Gerrit REST API[*1] を用いて，レビュー依頼した開発者がレビュー対象のソースコードに対して承認，要修正，コメントのいずれかの応答を行ったか否かを 2 値ラベルとして取得する．各レビュー依頼の時刻（レビュー依頼時刻）には，チケットの作成時刻を用いる．

時間分割：評価対象の開始時点 t_0 は，2023 年 01 月 01 日 00 時 00 分 00 秒とし，訓練期間は 3 年間，評価期間は 1 年

[*1] `https://review.opendev.org/`

表 2: データセット統計（bot 除外後）と評価期間別正例数（OpenStack・AOSP）

	OpenStack	AOSP
リポジトリ数	32	32
チケット（change）数	42,441	408,574
レビュー依頼行数	161,111	574,810
レビュアー数	2,372	8,901

	OpenStack		AOSP	
評価期間	正例数	正例率	正例数	正例率
W_1 (1–3m)	1,225	27.0%	7,607	28.4%
W_2 (4–6m)	1,021	22.5%	6,781	25.3%
W_3 (7–9m)	984	21.7%	6,188	23.1%
W_4 (10–12m)	811	17.9%	5,710	21.3%

間とする．**TODO**: [これは AOSP も同様？] なお，t_0 を単一時点に固定しているため，異なる時点での再現性は未検証であり，妥当性への脅威として 7 章で議論する．

データセットの規模：訓練期間と評価期間の両方で観測される (d, r) ペア $n = 4,534$（436 開発者）を分析対象とする．また，原型分析は t_0 直前 6 か月にレビュー・実装履歴がある 289 開発者のペア $n = 3,747$ に限定する．表 2 に bot 除外後のデータセット統計と，評価期間別の正例数（承諾ラベル）を OpenStack と AOSP をそれぞれ示す．

5.2 予測モデルの評価結果

表 3 と表 4 は，それぞれ開発者-ディレクトリのペア単位と開発者単位の IRL と RF の AUC-ROC の結果を，OpenStack・AOSP の両方について示す．各表では 4 つの評価期間を区別して示す．OpenStack のペア単位における両モデルの AUC-ROC の絶対値差は [0.1-5.2] パーセンテージポイント (以下，pp) 以内に収まり，開発者単位の両モデル AUC-ROC の絶対値差は [0.9-4.6] であった．全体として RF が上回る傾向ではあるが，IRL は RF と大きな精度差は見られなかった．AOSP もペア単位，開発者単位いずれも両モデルの差は最大 6.8pp（ペア単位，4–6m）にとどまり，OpenStack と同様に大きな精度差は見られなかった．

次節で両モデルが予測できる開発者の特徴を分析する．

5.3 時系列符号化の寄与を分離するアブレーション

IRL の精度が時系列情報の利用によるものか，IRL としての報酬関数の定式化によるものかを判別するため，同一のバックボーン **TODO**: [バックボーンって？]（LSTM，隠れ層次元 128），特徴量，学習データを用い，教師信号を最終時点のみに制限した比較モデルとの実験を行った．

表 5 に 4 評価期間を統合した AUC-ROC を示す．IRL 本体は，LSTM 版に対し +3.8 pp（bootstrap 95% CI $[+2.9, +4.7]$，$p < 0.0005$）と有意に上回った．また，比較モデル（LSTM 版）は RF に対しても有意に劣った（−5.0 pp，95% CI $[-5.8, -4.1]$）ことから，時系列モデルであれば精度が高いわけではなく，IRL の定式化が精度向上に寄与していると考えられる．

表 3: 開発者-ディレクトリペア単位 AUC-ROC（OpenStack $n = 4{,}534$；AOSP $n = 26{,}830$）

	OpenStack			AOSP		
評価期間	IRL	RF	diff	IRL	RF	diff
1–3 か月	0.826	0.791	+3.5	0.769	0.763	+0.6
4–6 か月	0.813	0.812	+0.1	0.693	0.761	−6.8
7–9 か月	0.751	0.792	−4.1	0.737	0.757	−2.0
10–12 か月	0.754	0.806	−5.2	0.711	0.765	−5.4
4 期間平均	**0.786**	**0.800**	**−1.4**	0.728	**0.762**	**−3.4**

表 4: 開発者単位 AUC-ROC（OpenStack $N = 436$；AOSP $N = 2{,}508$）

	OpenStack			AOSP		
評価期間	IRL	RF	diff	IRL	RF	diff
1–3 か月	0.873	0.882	−0.9	0.885	0.886	−0.1
4–6 か月	0.861	0.872	−1.1	0.877	0.884	−0.7
7–9 か月	0.826	0.871	**−4.5**	0.866	0.880	−1.4
10–12 か月	0.805	0.851	**−4.6**	0.868	0.884	−1.5
4 期間平均	0.841	**0.869**	**−2.8**	0.874	**0.883**	−0.9

表 5: IRL と比較モデルの AUC-ROC 比較（OpenStack・AOSP）

	OpenStack		AOSP	
モデル	AUC-ROC	IRL−X (pp)	AUC-ROC	IRL−X (pp)
IRL（提案手法）	**0.789**	—	**0.721**	—
LSTM 版	0.751	+3.8	実験中	実験中
RF	0.801	−1.2	0.762	−4.0

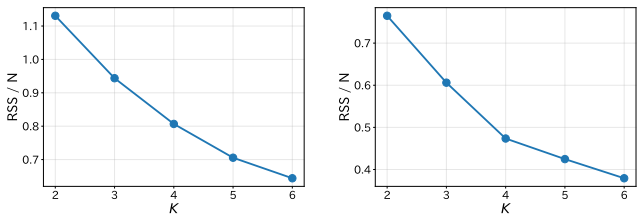
5.4 モデルが予測できるペア・開発者の分析結果

本節では，4 章で述べた原型分析を用いて，IRL と RF のそれぞれが予測できる開発者-ディレクトリのペアおよび開発者の特徴を分析する．原型分析以降は，OpenStack・AOSP のそれぞれについて分析結果を示す．

5.4.1 OpenStack の原型分析

分析対象は t_0 直前 6 か月にレビューや実装履歴を持つ開発者 289 人，およびそれらに関与するペア $n = 3,747$ ペアに限定する．図 3 は，原型数 K に対する平均残差平方和の推移を示す．原型数 $k = 3$ から $k = 4$ で平均残差平方和が大きく改善し $k = 5$ 以降は改善幅が小さくなったため $K = 4$ を採用する．

ペア単位の原型分析では，開発者とディレクトリを単位として分類する．特徴量は表 1 に示す 37 個の特徴量を用いる．表 6 と図 4 は，原型分析によって設定した 4 つの原型の解釈名，および，それぞれの原型の特徴量の分布を示す．表 6 に示す特徴の列は，図 4 の分析結果をまとめてい



(a) ペア単位（$n = 3{,}677$）　(b) 開発者単位（$N = 289$）

図 3: 原型数 K に対する平均残差平方和の推移．

る．図 4 の横軸は他の原型の平均との差，縦軸は特徴量名を示す．

表 7 にペア単位の 4 原型における評価期間別の AUC 差（IRL−RF）を示す．P1（他プロジェクト経験者）では 4 期間中 3 期間で IRL の方が精度が高く，当該ディレクトリへの貢献経験が少ないペアでは，時系列情報を考慮した IRL が有効である．P2（関与低下型）では短期（1–3 か月）においてのみ IRL がやや優位であり，中長期では RF が上回る．これは関与低下の初期段階における活動パターンの変化を時系列モデルが捉えることができたためと考える．P3（高承諾型）では，短期を除いて概ね RF が優位であり，累積レビュー実績が豊富な開発者-ディレクトリペアに対しては累積統計量を用いた RF が有効である．P4（低活動・低承諾型）では全期間にわたって RF が優位であり，活動が疎なペアでは IRL の学習に十分な行動軌跡が得られなかったことが理由と考える．

したがって，特定のディレクトリへの貢献経験が少ないペアでは IRL が，貢献経験が豊富なペアでは RF が有効であることを示した．特に，最も多くのペアを含む P1（$n = 1{,}284$）において IRL が優位であることは，提案手法の有効性を示す重要な結果である．

開発者単位の原型分析では，ペア単位とは異なり，ディレクトリとの関係性を考慮せず，開発者個別の活動特性に基づき分類する．特徴量は表 1 から，ディレクトリと活動特徴のカテゴリを除く 26 個の特徴量を用いる．表 8 と図 5 は，原型分析によって設定した 4 つの原型の解釈名，および，それぞれの原型の特徴量の分布を示す．表 9 に開発者単位の 4 原型における評価期間別の AUC 差（IRL−RF）を示す．

A1（低活動型）は，1–3 か月はほぼ互角（+0.1 pp）だが，それ以降の期間は RF の方が精度が高く，活動履歴が少ない開発者に対しては累積統計量を用いた RF が概ね優位である．一方，A2（短期・レビュー中心型）では 4–6 か月（+8.9 pp）や 10–12 か月（+16.2 pp）で IRL が上回る期間が見られるが，1–3 か月と 7–9 か月は RF が優位となり，評価期間によって優劣が混在する．A3（高承諾・高活動型）では 1–3 か月は IRL が上回るが（+7.5 pp），中長期は RF が優位であった（4 期間の合計差 −3.9 pp）．直近の行動変化を捉える IRL は短期予測に有効だが，中長期では一時的な変動が平均化され，安定した累積承諾実績を用いる RF が優位になると考える．A4（常時活動・低承諾型）は 4 期間中 3 期間で IRL が上回る（$\Sigma\Delta = +5.9$）．訓練期間全体を通じて活動月数（active_mo）が多いため，長い活動軌跡が IRL の学習を支えると考えられる．したがって，活動履歴が少ない開発者には累積統計量を用いた RF が有用であり，承諾パターンが不安定な開発者や活動軌跡が長い開発者では IRL が補完的に働く場合があることを示した．

5.4.2　AOSP の原型分析

図 6 は原型数 K に対する平均残差平方和の推移を示す．OpenStack と同様に $k = 4$ 以降で改善幅が小さくなるため，$K = 4$ を採用する．

表 11 と表 13 に，ペア単位・開発者単位の原型の解釈名と評価期間別 AUC を示す．AOSP でも両単位で IRL/RF の補完的な構造が確認された．ペア単位では，P4（高承諾・汎用型）で IRL が優位（$\Sigma\Delta = +14.3$）である一方，P1〜P3 では RF が優位であり，特に P2（ディレクトリ経験者・高関与型）で差が大きい（$\Sigma\Delta = -38.0$）．開発者単位では，A2（低活動型）と A3（高承諾・レビュアー専業型）で IRL が優位（$\Sigma\Delta = +8.3, +7.3$），A1（常時活動・高承諾型）と A4（低承諾型）で RF が優位である．

ペア単位では，AOSP の P4 は対象ディレクトリへの直接レビュー経験（path_n_rev= 0.017）が薄いタイプで IRL が優位であった．ただし，同じく経験が薄い P1（path_n_rev= 0.000，4 原型中最小）では RF が優位（$\Sigma\Delta = -11.6$）であり，ディレクトリ経験の薄さ単独では両モデルの優劣を説明できない．一方，最もディレクトリ経験・関与が高い P2（ディレクトリ経験者・高関与型）では RF が大きく優位であり（$\Sigma\Delta = -38.0$），高い関与度が RF に有利に働くという OpenStack と整合する傾向も確認された．開発者単位では OpenStack の A1（低活動型）が RF 優位であったのに対し，AOSP の A2（低活動型）は IRL が優位という対照的な結果が得られた．AOSP では低活動な開発者ほど活動の変動情報が RF の累積統計より有効に働く可能性を示唆するが，原因の特定には追加の分析が必要である．

5.5　追加実験：アンサンブルモデル

5.4 節で確認した IRL と RF は，予測可能な開発者とディレクトリのペア，開発者が異なるため，モデル間で補完することで高い精度が期待できる．追加実験として両モデルの貢献確率の単純平均をアンサンブルの予測値とし，予測精度の改善を検証する．改善幅が偶然でないことを確認するために，bootstrap 法により 95%信頼区間を算出して判断する．

表 14 にアンサンブルモデルの結果を示す．全体では最良単独モデル（IRL と RF のうち AUC-ROC が高い方）と比

表 6: 4 ペア archetype の解釈名

ID	名前	特徴
P1	他プロジェクト経験者	他プロジェクトへの貢献経験（cross_proj）を有し，最終活動からの経過時間（path_recency）が短く，ディレクトリへの過去レビュー数（path_n_rev）は少ない
P2	関与低下型	当該ディレクトリの最終活動からの経過時間（path_recency）が長い
P3	高承諾型	承諾率（accept_rate_all 等）が高い
P4	低活動・低承諾型	全体承諾率（accept_rate_all）が低く，活動頻度（act_freq）も少ない

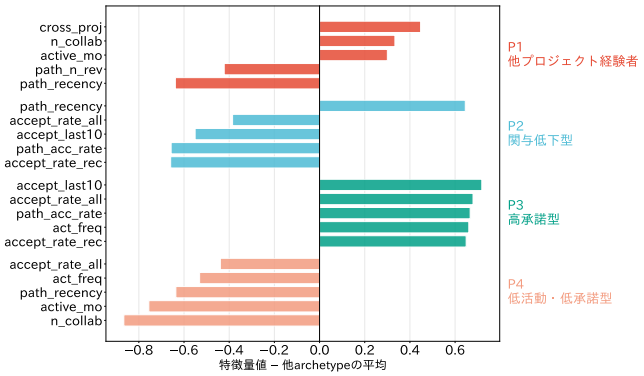
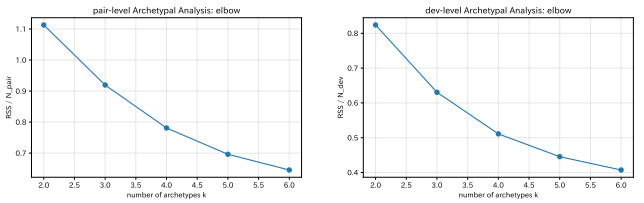


図 4: ペア単位の原型分析における特徴的な特徴量

表 7: ペア単位の 4 原型における評価期間別 AUC

		1–3m	4–6m	7–9m	10–12m	ΣΔ
P1 (1,284)	IRL	0.765	0.742	0.706	0.654	
	RF	0.675	0.703	0.664	0.708	
	Δ	+9.0	+3.9	+4.2	−5.4	**+11.7**
P2 (928)	IRL	0.745	0.783	0.724	0.716	
	RF	0.726	0.798	0.725	0.758	
	Δ	+2.0	−1.5	−0.1	−4.2	−3.8
P3 (539)	IRL	0.800	0.714	0.659	0.752	
	RF	0.742	0.737	0.729	0.759	
	Δ	+5.8	−2.3	−7.0	−0.7	−4.2
P4 (926)	IRL	0.692	0.682	0.643	0.630	
	RF	0.696	0.687	0.730	0.688	
	Δ	−0.4	−0.5	−8.7	−5.9	−15.5



(a) ペア単位（$n = 21{,}291$）　(b) 開発者単位（$N = 1{,}248$）

図 6: AOSP：原型数 K に対する平均残差平方和

表 8: 4 原型の解釈名

ID	名前	特徴
A1	低活動型	活動頻度（act_freq）・活動月数（active_mo）・コード作成に対するレビュー比率（rev/owner）が低く，同一組織からの依頼が多い（same_domain）
A2	短期・レビュー中心型	コード作成に対するレビュー貢献率（rev/owner）が高く，活動月数（active_mo）や承諾率（accept_rate_all）が低い
A3	高承諾・高活動型	直近の承諾率（accept_last10，accept_rate_rec）や活動頻度（act_freq）が高い
A4	常時活動・低承諾型	活動月数（active_mo）が長いが，承諾率（accept_rate_all 等）が低い

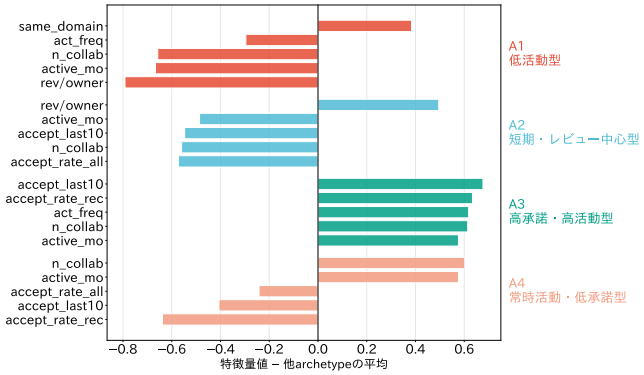


図 5: 開発者単位の原型分析における特徴的な特徴量

表 9: 開発者単位の 4 原型における評価期間別 AUC

		1–3m	4–6m	7–9m	10–12m	ΣΔ
A1 (341)	IRL	0.607	0.423	0.630	0.492	
	RF	0.606	0.631	0.651	0.597	
	Δ	+0.1	−20.8	−2.1	−10.5	−33.3
A2 (554)	IRL	0.750	0.806	0.667	0.664	
	RF	0.798	0.718	0.756	0.502	
	Δ	−4.9	+8.9	−8.9	+16.2	**+11.3**
A3 (1,213)	IRL	0.835	0.781	0.693	0.730	
	RF	0.760	0.782	0.741	0.796	
	Δ	+7.5	−0.1	−4.8	−6.5	−3.9
A4 (1,639)	IRL	0.791	0.832	0.763	0.773	
	RF	0.746	0.814	0.747	0.793	
	Δ	+4.5	+1.8	+1.6	−2.0	**+5.9**

較して +2.3 pp（95% CI [+1.7, +2.8]）の改善を確認した．評価期間別では短中期（1–3, 4–6 か月）信頼区間が 0 を含まない改善が得られたが，長期（7–9, 10–12 か月）では改善幅が小さく 95%信頼区間が 0 を含む．表 14 下段に AOSP の結果を示す．全体では +0.7 pp（95% CI [+0.5, +1.0]）の改善を確認したが，OpenStack と異なり 4–6 か月では信頼区間が 0 を含まない悪化（−0.6 pp，95% CI [−1.0, −0.2]）が見られた．1–3，7–9 か月は有意な改善が得られた一方，10–12 か月は信頼区間が 0 を含み有意ではなかった．

表 10: AOSP：ペア単位の原型の解釈名

ID	名前	特徴
P1	高活動・未経験ディレクトリ型	活動頻度（act_freq）・他プロジェクト経験（cross_proj）・協力者数（n_collab）が高い一方，対象ディレクトリの直近活動（path_recency）が低い
P2	ディレクトリ経験者・高関与型	対象ディレクトリの直近活動（path_recency）・過去レビュー数（path_n_rev）に加え，活動頻度（act_freq）・協力者数（n_collab）も高い
P3	低承諾型	直近承諾率（accept_last10）・全体承諾率（accept_rate_all）・活動月数（active_mo）・協力者数（n_collab）がいずれも低い
P4	高承諾・汎用型	直近承諾率（accept_last10）・全体承諾率（accept_rate_all）が高い一方，活動頻度（act_freq）は低い

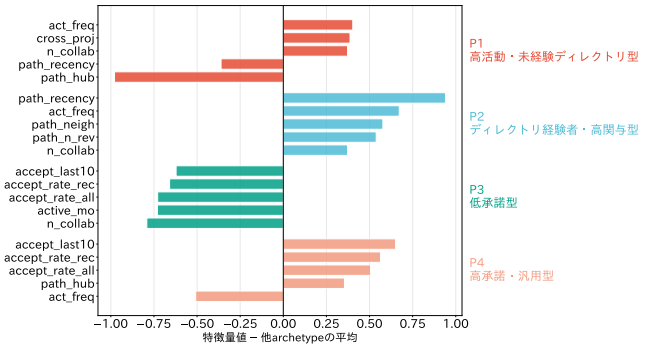


図 7: ペア単位の原型分析における特徴的な特徴量 **TODO**: [AOSP？？]

表 11: ペア単位の 4 原型における評価期間別 AUC **TODO**: [AOSP？？]

		1–3m	4–6m	7–9m	10–12m	ΣΔ
P1 (–)	IRL	0.759	0.733	0.742	0.660	
	RF	0.755	0.765	0.744	0.746	
	Δ	+0.4	−3.3	−0.2	−8.6	−11.6
P2 (–)	IRL	0.734	0.584	0.595	0.594	
	RF	0.777	0.698	0.694	0.718	
	Δ	−4.4	−11.4	−9.8	−12.4	−38.0
P3 (–)	IRL	0.829	0.792	0.735	0.768	
	RF	0.814	0.797	0.764	0.793	
	Δ	+1.5	−0.5	−2.9	−2.6	−4.6
P4 (–)	IRL	0.772	0.721	0.771	0.773	
	RF	0.726	0.699	0.732	0.738	
	Δ	+4.6	+2.3	+3.9	+3.5	**+14.3**

表 12: AOSP：開発者単位の原型の解釈名

ID	名前	特徴
A1	常時活動・高承諾型	協力者数（n_collab）・活動月数（active_mo）・活動頻度（act_freq）・他プロジェクト経験（cross_proj）がいずれも高い
A2	低活動型	同一組織からの依頼（same_domain）・協力者数（n_collab）・活動月数（active_mo）・レビュー比率（rev/owner）がいずれも低い
A3	高承諾・レビュアー専業型	直近承諾率（accept_last10）・全体承諾率（accept_rate_all）・レビュー比率（rev/owner）が高い
A4	低承諾型	レビュー比率（rev/owner）は高いが，直近承諾率（accept_last10）・全体承諾率（accept_rate_all）は低い

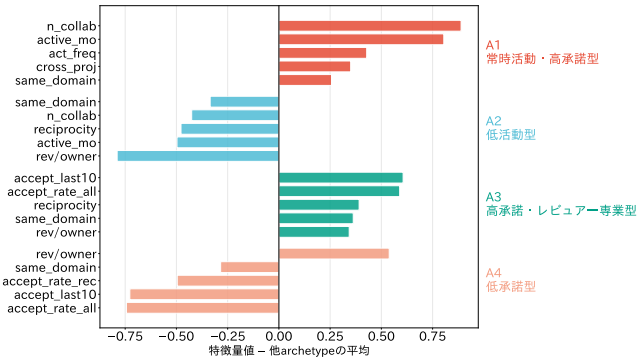


図 8: 開発者単位の原型分析における特徴的な特徴量 **TODO**: [AOSP]

表 13: 開発者単位の 4 原型における評価期間別 AUC **TODO**: [AOSP]

		1–3m	4–6m	7–9m	10–12m	ΣΔ
A1 (–)	IRL	0.767	0.657	0.701	0.683	
	RF	0.774	0.750	0.751	0.764	
	Δ	−0.7	−9.3	−4.9	−8.1	−23.0
A2 (–)	IRL	0.812	0.786	0.782	0.804	
	RF	0.790	0.776	0.765	0.770	
	Δ	+2.2	+1.1	+1.7	+3.4	+8.3
A3 (–)	IRL	0.774	0.771	0.749	0.816	
	RF	0.759	0.762	0.764	0.752	
	Δ	+1.5	+0.9	−1.5	+6.4	+7.3
A4 (–)	IRL	0.842	0.828	0.758	0.789	
	RF	0.830	0.822	0.770	0.814	
	Δ	+1.3	+0.6	−1.2	−2.6	−1.8

表 15 にペア単位の 4 原型における評価期間別のアンサンブルモデルの最良単独モデルからの改善精度を示す．ペア単位の 4 種類（P1，P2，P3，P4）の評価期間別の 4 種類（合計 16 種類）のうち，13 種類でアンサンブルモデルが最良単独モデルから改善されることを示した（最大 +3.6 pp，P3 10–12m）．特に，P1（他プロジェクト経験者）は全期間で改善した（+0.4〜+2.2 pp）．表 16 に開発者単位の 4 原型における評価期間別のアンサンブルモデルの改善幅を示す．A3（高承諾・高活動型）と A4（常時活動・低承諾型）の全期間で正の改善した一方，A1（低活動型）と

表 14: IRL，RF，アンサンブルによるモデルの精度比較結果と，bootstrap による 95%信頼区間

OpenStack					
期間	IRL	RF	アンサンブル	改善幅 (pp)	95% CI
1–3m	0.826	0.791	0.838	+1.2	[+0.8, +1.8]
4–6m	0.813	0.812	0.842	+2.9	[+2.3, +3.6]
7–9m	0.751	0.792	0.799	+0.6	[−0.5, +1.7]
10–12m	0.754	0.806	0.808	+0.2	[−0.7, +1.3]
全体	0.789	0.801	**0.823**	**+2.3**	[+1.7, +2.8]
AOSP					
期間	IRL	RF	アンサンブル	改善幅 (pp)	95% CI
1–3m	0.769	0.763	0.789	+2.0	[+1.7, +2.3]
4–6m	0.693	0.761	0.755	−0.6	[−1.0, −0.2]
7–9m	0.737	0.757	0.773	+1.6	[+1.1, +2.1]
10–12m	0.711	0.765	0.764	−0.1	[−0.6, +0.4]
全体	0.721	0.762	**0.769**	**+0.7**	[+0.5, +1.0]

表 15: ペア単位の 4 原型におけるアンサンブルモデルを用いた評価期間別の AUC 差（アンサンブル − 最良単独）

OpenStack					
ペア単位 (n)	1–3m	4–6m	7–9m	10–12m	合計
P1 (1,284)	+0.4	+2.2	+1.7	+1.2	+5.5
P2 (928)	+0.9	+1.3	+1.3	−0.1	+3.4
P3 (539)	+2.5	+0.8	+0.2	+3.6	+7.1
P4 (926)	+2.8	+3.0	−3.3	−1.3	+1.2
AOSP					
ペア単位 (n)	1–3m	4–6m	7–9m	10–12m	合計
P1 (5,068)	+2.0	+0.0	+2.7	−1.5	+3.2
P2 (4,422)	−0.2	−2.7	−2.9	−6.3	−12.1
P3 (2,878)	+0.6	+1.6	+1.9	+1.6	+5.8
P4 (8,923)	+0.4	+2.2	+1.7	+1.9	+6.1

A2（短期・レビュー中心型）では改善しない期間も確認した．表 15・表 16 下段に AOSP の結果を示す．最も多くのペアを含む P4（$n = 8{,}923$）が 3 期間で正の改善を示した（合計 +6.1 pp）一方，P2 は全期間で悪化した（合計 −12.1 pp）．開発者単位では A3 が全期間で改善した（合計 +4.7 pp）一方，A1 は全期間で悪化した（合計 −8.3 pp）．

以上より，ペア数が最も多い P1，開発者数が多い A3・A4 は全期間にわたって改善を示したのは，アンサンブルモデルの有効性を示す結果である．本研究では単純平均によるアンサンブルを採用したが，原型や評価期間に応じてモデルを選択する方法を導入することで，さらなる精度改善が期待できる．

6. 考察

6.1 IRL と RF の補完性に関する考察

OpenStack：5.4 節で観察されたモデル間の補完性は，開発者の原型ごとに行動パターンが異なることによる結果と

表 16: 開発者単位の 4 原型におけるアンサンブルモデルを用いた評価期間別の AUC 差 (アンサンブル − 最良単独)

OpenStack					
開発者単位 (n)	1–3m	4–6m	7–9m	10–12m	合計
A1 (341)	+2.8	−9.3	+1.7	−2.5	−7.3
A2 (554)	+2.0	−0.4	−4.4	−12.6	−15.4
A3 (1,213)	+1.6	+3.9	+2.1	+0.9	+8.5
A4 (1,639)	+0.4	+1.7	+1.4	+1.6	+5.1
AOSP					
開発者単位 (n)	1–3m	4–6m	7–9m	10–12m	合計
A1 (483)	−2.0	−1.2	−2.0	−3.0	−8.3
A2 (258)	−0.8	+0.1	−1.2	+1.3	−0.6
A3 (225)	+0.2	−0.5	+2.0	+3.1	+4.7
A4 (282)	+0.5	+0.8	−0.2	+0.3	+1.4

考える．A1（低活動型）は，貢献数が少なく行動軌跡が短いため，IRL で開発者の行動パターンを十分に学習できなかったことが考えられる．一方，RF は累積活動量が少ないことが，活動停止を予測する特徴量としてモデルに寄与するため，3 か月以降の評価期間では IRL よりも高い精度で予測できた．1–3 か月では両モデルの精度が拮抗しており（IRL 0.607，RF 0.606），他の原型と比べて低い水準にとどまった．

A2（短期・レビュー中心型）では，累積統計量のみを用いる RF は訓練期間の低い承諾率をそのまま反映するため，評価期間の行動変化を捉えることが難しいが，月次の行動軌跡を通じて状態変化を取り込める IRL は部分的な行動変化を捉えられたと考える．なお，A2 では IRL と RF の優劣が評価期間によって入れ替わっており，これは OpenStack が半年ごとにリリースを行うため，リリースサイクルに伴う活動パターンの変化が影響していることも考えられる．

A3（高承諾・高活動型）では，貢献率が高く，活動も活発で行動パターンが安定しているため，時系列的な変動を考慮せずとも類型統計量を用いる RF が有効であったと考える．ただし，1–3 か月では IRL が優位であった．これは他の期間に比べて直近の活発な行動パターンを IRL が捉えたことが原因と考える．

A4（常時活動・低承諾型）では，活動月数が多く，行動履歴が豊富なため，両モデルとも安定した精度を示し，優劣の差は小さくなったと考える．両モデルともアンサンブルの改善幅が全期間で正（A3: +0.9〜+3.9 pp，A4: +0.4〜+1.7 pp）となり，両モデルでほぼ同じ精度であった．しかし，IRL が誤予測するペアと RF が誤予測するペアが異なるため，単純平均によるアンサンブルの効果が安定して示された．開発者の原型とペアの原型で補完性の傾向が異なる．ペアの原型では，貢献経験が少ないペア（P1 他プロジェクト経験者）は特定のディレクトリの累積統計量が少ないため，RF よりも IRL が有効であったと考える．一方で，ディレクトリへの貢献経験を有するペア（P2 関与

低下型，P3 高承諾型）では累積統計量が有効に寄与する RF の精度が高くなったと考える．

AOSP：5.4.2 節で示した通り，ペア単位・開発者単位いずれも OpenStack と同様の補完性が確認されたが，各原型で IRL と RF のどちらが優位かを単一の特徴量のみで説明することは難しい．

6.2 アンサンブルの有効性に関する考察

OpenStack：両モデルの予測確率の単純平均が，精度向上につながった理由を考察する．第一に，両モデルが異なる開発者タイプで予測を外していることが考えられる．A1 では RF が概ね優位（最大 −20.8 pp），A2 では一部の評価期間（4–6, 10–12 か月）で IRL が優位となる傾向が見られた．このように，IRL の予測結果が誤るペアと，RF の予測結果が誤るペアとが異なるため，単純平均でも誤予測が部分的に相殺され，全体として精度が改善したと考える．

第二に，ペアの原型と評価期間の 16 通りの組み合わせのうち，13 通りでアンサンブルの改善がみられ（最大 +3.6 pp），特に P1（他プロジェクト経験者）では全期間で改善した．これは 5.4.1 節で示した補完性がアンサンブルの精度改善につながったと考える．

AOSP：5.5 節で示した通り，全体では有意な改善が得られたが，開発者単位の A1（常時活動・高承諾型）ではアンサンブルが全期間で悪化した（合計 −8.3 pp）．A1 は活動が非常に活発なため，RF の累積統計量のみで十分に高い精度が得られる一方（4 期間平均 0.760），IRL は 10–12 か月に RF へ −8.1 pp の差をつけられており，長期になるほど時系列情報の優位性が薄れる開発者に対しては単純平均のアンサンブルがかえって精度を落とすと考えられる．

7. 妥当性への脅威

7.1 内的妥当性

訓練時に評価期間の情報が混入することが懸念されるが，ベースモデルは訓練期間と評価期間を時系列に分離して学習し，未来の情報を学習することがないよう設計している．アンサンブルモデルにおいても，単純平均であるため統合段階で未来の情報を学習することはない．原型分析は評価集合の特徴量から原型と重み $\boldsymbol{\alpha}$ を計算しているが，補完性の事後解釈にのみ用いておりモデルの訓練・評価とは独立しているため AUC には影響しない．

7.2 外的妥当性

評価対象は Gerrit を用いる 2 つの OSS プロジェクト（OpenStack・AOSP）に限定しているため，GitHub の Pull Request ベースのワークフローを採るプロジェクトなど，レビュー単位や承諾の表現粒度が異なるプラットフォームへの一般化は今後の課題とする．ただし，アンサンブルの効果は，累積統計量を用いる RF と時系列情報を用いる IRL が互いに異なる誤りを犯すという構造に支えられており，この構造は特定のレビュー基盤に強く依存しないと考えられる．実際，独立した 2 つのエコシステムの両方でこの構造に基づく改善を確認できたことは，この解釈を支持する．スナップショット時刻 t_0 を 2023 年 1 月 1 日に固定しているため，異なる日時での再現は未検証である．ただし，OpenStack は半年ごとに新バージョンをリリースするため，訓練期間（3 年間）および評価期間（1 年間）にはリリース前後の活動量の変動が複数回含まれる．したがって，リリースサイクルに依存した偏りは分析期間内にも存在し，単一の t_0 であっても分析として結果の信頼性は確保されていると考える．

8. おわりに

本研究は，特定のモジュールに対して特定の開発者がレビューに貢献するか否かの予測に対し，時系列情報を用いる IRL と累積統計量を用いる RF のモデルを構築した．ケーススタディとして，OpenStack の公式サービスリポジトリ 32 件，および AOSP の 32 リポジトリを対象に実験を行った．原型分析で分解した開発者のタイプ間では，両方のモデルは予測結果を補完しあうことで精度向上が期待されることを明らかにし，アンサンブルモデルを提案した．その結果，それぞれのモデルの最大値に比べ，OpenStack で平均 +2.3 pp，AOSP で平均 +0.7 pp の改善が見られた．今後は，IRL が推定した報酬関数を環境のフィードバックとして組み込み，開発者の継続性を考慮したレビュー依頼の推薦方策の設計と評価へ展開する．

謝辞　本研究は JSPS 科研費 25K15058 の助成を受けたものです．

参考文献

[1] Ye, Y. and Kishida, K.: Toward an understanding of the motivation of open source software developers, *Proceedings of the International Conference on Software Engineering (ICSE)*, IEEE, pp. 419–429 (2003).

[2] Raymond, E.: The cathedral and the bazaar, *Knowledge, Technology & Policy*, Vol. 12, No. 3, pp. 23–49 (1999).

[3] Howison, J., Inoue, K. and Crowston, K.: Social dynamics of free and open source team communications, *IFIP International Conference on Open Source Systems*, Springer, pp. 319–330 (2006).

[4] Pinto, G., Steinmacher, I. and Gerosa, M. A.: More common than you think: An in-depth study of casual contributors, *Proceedings of the 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, IEEE, pp. 112–123 (2016).

[5] Bacchelli, A. and Bird, C.: Expectations, outcomes, and challenges of modern code review, *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, IEEE, pp. 712–721 (2013).

[6] McIntosh, S., Kamei, Y., Adams, B. and Hassan, A. E.: An empirical study of the impact of modern code review

practices on software quality, *Empirical Software Engineering*, Vol. 21, No. 5, pp. 2146–2189 (2016).

[7] Zhou, M. and Mockus, A.: What make long term contributors: Willingness and opportunity in OSS community, *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, IEEE, pp. 518–528 (2012).

[8] Bao, L., Xia, X., Lo, D. and Murphy, G. C.: A large scale study of long-time contributor prediction for GitHub projects, *IEEE Transactions on Software Engineering*, Vol. 47, No. 6, pp. 1277–1298 (2019).

[9] Constantinou, E. and Mens, T.: An empirical comparison of developer retention in the RubyGems and npm software ecosystems, *Innovations in Systems and Software Engineering*, Vol. 13, pp. 101–115 (2017).

[10] Ruangwan, S., Thongtanunam, P., Ihara, A. and Matsumoto, K.: The impact of human factors on the participation decision of reviewers in modern code review, *Empirical Software Engineering*, Vol. 24, No. 2, pp. 973–1016 (2019).

[11] Iaffaldano, G., Steinmacher, I., Calefato, F., Gerosa, M. and Lanubile, F.: Why do developers take breaks from contributing to OSS projects? A preliminary analysis, *arXiv preprint arXiv:1903.09528* (2019).

[12] Calefato, F., Gerosa, M. A., Iaffaldano, G., Lanubile, F. and Steinmacher, I.: Will you come back to contribute? Investigating the inactivity of OSS core developers in GitHub, *Empirical Software Engineering*, Vol. 27, No. 3, p. 76 (2022).

[13] Vasilescu, B., Posnett, D., Ray, B., van den Brand, M. G. J., Serebrenik, A., Devanbu, P. T. and Filkov, V.: Gender and tenure diversity in GitHub teams, *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI)*, pp. 3789–3798 (2015).

[14] Lin, B., Robles, G. and Serebrenik, A.: Developer turnover in global, industrial open source projects: Insights from applying survival analysis, *Proceedings of the IEEE 12th International Conference on Global Software Engineering (ICGSE)*, IEEE, pp. 66–75 (2017).

[15] Liu, Y., Qi, X., Zhang, J., Li, H., Ge, X. and Ai, J.: Automatic bug triaging via deep reinforcement learning, *Applied Sciences*, Vol. 12, No. 7, p. 3565 (2022).

[16] Elkan, C. and Noto, K.: Learning classifiers from only positive and unlabeled data, *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 213–220 (2008).

[17] Ziebart, B. D., Maas, A., Bagnell, J. A. and Dey, A. K.: Maximum entropy inverse reinforcement learning, *Proceedings of the 23rd AAAI Conference on Artificial Intelligence (AAAI)*, pp. 1433–1438 (2008).

[18] Kingma, D. P. and Ba, J.: Adam: A method for stochastic optimization, *arXiv preprint arXiv:1412.6980* (2014).

[19] Breiman, L.: Random forests, *Machine Learning*, Vol. 45, No. 1, pp. 5–32 (2001).

[20] Cutler, A. and Breiman, L.: Archetypal analysis, *Technometrics*, Vol. 36, No. 4, pp. 338–347 (1994).